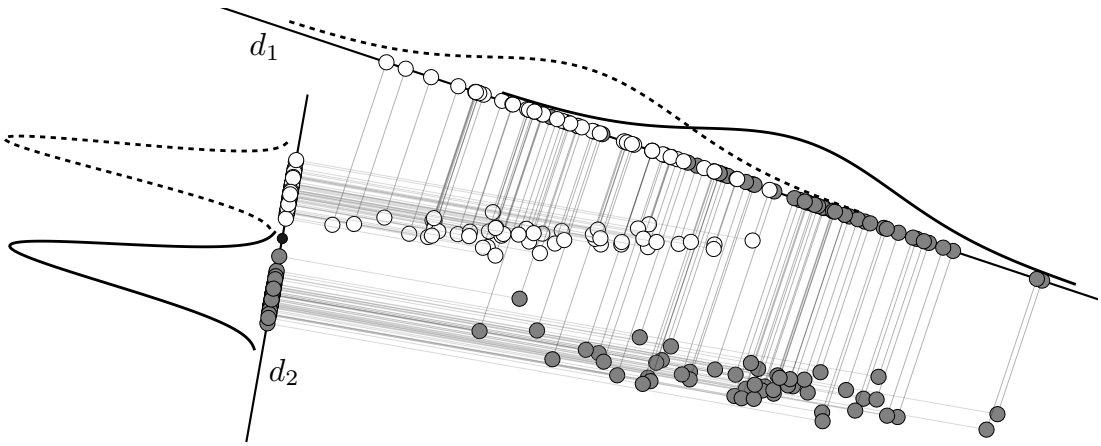
Chương 22

Phân tích biệt thức tuyến tính

22.1. Giới thiệu

Trong chương trước, chúng ta đã làm quen với một thuật toán giảm chiều dữ liệu phổ biến nhất – phân tích thành phần chính (PCA). Như đã đề cập, PCA là một thuật toán học không giám sát, tức chỉ sử dụng các vector dữ liệu mà không cần tới nhãn. Tuy nhiên, trong bài toán phân loại, việc khai thác mối liên quan giữa dữ liệu và nhãn sẽ mang lại kết quả phân loại tốt hơn.

Nhắc lại rằng PCA là phương pháp giảm chiều dữ liệu sao cho lượng thông tin về dữ liệu giữ lại, thể hiện ở tổng phương sai của các thành phần giữ lại, là nhiều nhất. Tuy nhiên, trong nhiều bài toán, ta không cần giữ lại lượng thông tin lớn nhất mà chỉ cần giữ lại thông tin cần thiết cho riêng bài toán đó. Xét ví dụ bài toán phân loại nhị phân được minh họa trong Hình 22.1. Ở đây, giả sử rằng dữ liệu được chiếu lên một đường thẳng và mỗi điểm được thay bởi hình chiếu của nó lên đường thẳng kia. Như vậy, số chiều dữ liệu đã được giảm từ hai về một. Câu hỏi đặt ra là đường thẳng cần có phương như thế nào để hình chiếu của dữ liệu có ích cho việc phân loại nhất? Việc phân loại trong không gian một chiều có thể hiểu là việc tìm ra một ngưỡng giúp phân tách hai lớp. Xét hai đường thẳng d_1 và d_2. Trong đó phương của d_1 gần với phương của thành phần chính nếu thực hiện PCA, phương của d_2 gần với phương của thành phần phụ tìm được bằng PCA. Nếu thực hiện giảm chiều dữ liệu bằng PCA, ta sẽ thu được dữ liệu gần với các điểm được chiếu lên d_1. Lúc này việc phân tách hai lớp trở nên phức tạp vì các điểm dữ liệu mới của hai lớp chồng lấn lên nhau. Ngược lại, nếu ta chiếu dữ liệu lên đường thẳng gần với thành phần phụ tìm được bởi PCA, tức d_2, các điểm hình chiếu nằm hoàn toàn về hai phía khác nhau của điểm màu đen trên đường thẳng này. Như vậy, việc chiếu dữ liệu lên d_2 sẽ mang lại hiệu quả hơn trong bài toán phân loại. Việc phân loại một điểm dữ liệu mới được xác định nhanh chóng bằng cách so sánh hình chiếu của nó lên d_2 với điểm phân ngưỡng màu đen.



Hình 22.1. Chiếu dữ liệu lên các đường thẳng khác nhau. Có hai lớp dữ liệu minh hoạ bởi các điểm màu xám và trắng trong không gian hai chiều. Số chiều được giảm về một bằng cách chiếu dữ liệu lên các đường thẳng khác nhau d_1 và d_2. Trong hai cách chiếu này, phương của d_1 gần giống với phương của thành phần chính thứ nhất của dữ liệu, phương của d_2 gần với thành phần phụ của dữ liệu nếu dùng PCA. Khi chiếu dữ liệu lên d_1, nhiều điểm màu xám và trắng bị chồng lấn lên nhau, khiến cho việc phân loại dữ liệu là không khả thi trên đường thẳng này. Ngược lại, khi được chiếu lên d_2, dữ liệu của hai lớp được chia thành các cụm tương ứng tách biệt nhau, khiến cho việc phân loại trở nên đơn giản và hiệu quả hơn. Các đường cong hình chuông thể hiện xấp xỉ phân bố xác suất của dữ liệu hình chiếu trong mỗi lớp.

Qua ví dụ trên ta thấy rằng, việc giữ lại thông tin nhiều nhất không mang lại kết quả tốt trong một số trường hợp. Chú ý rằng kết quả của phân tích trên đây không có nghĩa là thành phần phụ mang lại hiệu quả tốt hơn thành phần chính. Việc chiếu dữ liệu lên đường thẳng nào giúp ích cho các bài toán phân loại cần nhiều phân tích cụ thể hơn. Ngoài ra, hai đường thẳng d_1 và d_2 trên đây không vuông góc với nhau, chúng được chọn gần với các thành phần chính và phụ của dữ liệu phục vụ cho mục đích minh hoạ.

Phân tích biệt thức tuyến tính (linear discriminant analysis, LDA) ra đời giúp tìm phương chiếu dữ liệu hiệu quả cho bài toán phân loại. LDA có thể được coi là một phương pháp giảm chiều dữ liệu hoặc phân loại, hoặc được áp dụng đồng thời cho cả hai, tức giảm chiều dữ liệu sao cho việc phân loại hiệu quả nhất. Số chiều của dữ liệu mới nhỏ hơn hoặc bằng $C - 1$ trong đó C là số lớp dữ liệu. Từ *biệt thức* (discriminant) được hiểu là *những thông tin riêng biệt của mỗi lớp*.

Trong Mục 22.2 dưới đây, chúng ta sẽ thảo luận về LDA cho bài toán phân loại nhị phân. Mục 22.3 sẽ tổng quát LDA lên cho trường hợp phân loại đa lớp. Mục 22.4 trình bày các ví dụ và mã nguồn Python cho LDA.

22.2. Bài toán phân loại nhị phân

22.2.1. Ý tưởng cơ bản

Quay lại với Hình 22.1, giả sử dữ liệu của mỗi lớp khi chiếu xuống một đường thẳng tuân theo phân phối chuẩn có hàm mật độ xác suất dạng hình chuông. Độ rộng của mỗi đường hình chuông này thể hiện *độ lệch chuẩn* (standard deviation[56], ký hiệu là s) của dữ liệu. Dữ liệu càng tập trung thì độ lệch chuẩn càng nhỏ, càng phân tán thì độ lệch chuẩn càng cao. Khi chiếu lên d_1, dữ liệu của hai lớp bị phân tán quá nhiều và trộn lẫn vào nhau. Khi chiếu lên d_2, mỗi lớp có độ lệch chuẩn nhỏ, khiến dữ liệu trong từng lớp tập trung hơn.

Tuy nhiên, việc độ lệch chuẩn nhỏ trong mỗi lớp chưa đủ để đảm bảo độ độ riêng biệt của dữ liệu giữa hai lớp. Xét các ví dụ trong Hình 22.2. Hình 22.2a thể hiện trường hợp dữ liệu được chiếu lên d_1 như trong Hình 22.1. Cả hai lớp đều quá phân tán khiến lượng chồng lấn lớn (phần diện tích màu xám), tức dữ liệu chưa thực sự riêng biệt. Hình 22.2b thể hiện trường hợp độ lệch chuẩn của hai lớp đều nhỏ, tức dữ liệu trong mỗi lớp tập trung hơn. Tuy nhiên, khoảng cách giữa hai lớp, được đo bằng khoảng cách giữa hai trung bình m_1 và m_2 là quá nhỏ. Việc này khiến phần chồng lấn chiếm một tỉ lệ lớn, không tốt cho việc phân loại. Hình 22.2c thể hiện trường hợp cả hai độ lệch chuẩn nhỏ và khoảng cách giữa hai trung bình lớn, phần chồng lấn nhỏ không đáng kể.

Vậy, độ lệch chuẩn và khoảng cách giữa hai trung bình cụ thể đại diện cho các tiêu chí gì?

- Độ lệch chuẩn nhỏ thể hiện việc dữ liệu ít phân tán, tức dữ liệu trong mỗi lớp có xu hướng giống nhau. Hai phương sai s_1^2, s_2^2 còn được gọi là các *phương sai nội lớp* (within-class variance).

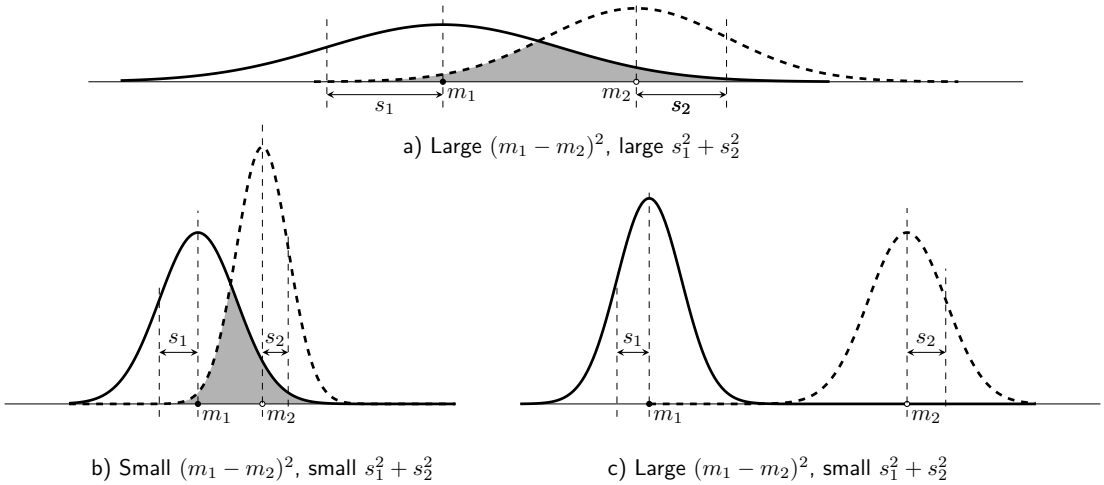
- Khoảng cách giữa các trung bình lớn chứng tỏ hai lớp cách xa nhau, tức dữ liệu giữa hai lớp khác nhau nhiều. Bình phương khoảng cách giữa hai trung bình $(m_1 - m_2)^2$ còn được gọi là *phương sai liên lớp* (between-class variance). Ở đây bình phương của hiệu hai trung bình được lấy để có cùng thứ nguyên với phương sai.

Hai lớp dữ liệu được gọi là tách biệt (discriminative) nếu hai lớp đó cách xa nhau (phương sai liên lớp lớn) và dữ liệu trong mỗi lớp có xu hướng giống nhau (phương sai nội lớp nhỏ). LDA là thuật toán đi tìm một phép chiếu sao cho tỉ lệ giữa phương sai liên lớp và phương sai nội lớp trở nên lớn nhất có thể.

22.2.2. Hàm mục tiêu của phân tích biệt thức tuyến tính

Giả sử có N điểm dữ liệu $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N \in \mathbb{R}^D$ trong đó $N_1 < N$ điểm đầu tiên thuộc lớp thứ nhất, $N_2 = N - N_1$ điểm còn lại thuộc lớp thứ hai. Ký

[56] độ lệch chuẩn là căn bậc hai của phương sai



Hình 22.2. Khoảng cách giữa các trung bình và tổng các phương sai ảnh hưởng tới độ biệt thức của dữ liệu. (a) Khoảng cách giữa hai trung bình lớn nhưng phương sai trong mỗi class cũng lớn, khiến cho hai phân phối chồng lấn lên nhau (phần màu xám). (b) Phương sai cho mỗi class rất nhỏ nhưng hai trung bình quá gần nhau, khiến khó phân biệt hai class. (c) Khi phương sai đủ nhỏ và khoảng cách giữa hai trung bình đủ lớn, ta thấy rằng dữ liệu hai lớp tách biệt hơn.

hiệu $\mathcal{C}_1 = \{n|1 \leq n \leq N_1\}$ là tập hợp chỉ số các điểm thuộc lớp thứ nhất và $\mathcal{C}_2 = \{m|N_1 + 1 \leq m \leq N\}$) là tập hợp chỉ số các điểm thuộc lớp thứ hai. Phép chiếu dữ liệu xuống một đường thẳng được mô tả bằng một vector trọng số $\mathbf{w}$, giá trị tương ứng của mỗi điểm dữ liệu chiếu được cho bởi

$$z_n = \mathbf{w}^T\mathbf{x}_n, 1 \leq n \leq N. \tag{22.1}$$

Vector trung bình của mỗi lớp được tính bởi

$$\mathbf{m}_k = \frac{1}{N_k}\sum_{n\in\mathcal{C}_k}\mathbf{x}_n, \quad k = 1, 2 \tag{22.2}$$

$$\Rightarrow m_1 - m_2 = \frac{1}{N_1}\sum_{i\in\mathcal{C}_1} z_i - \frac{1}{N_2}\sum_{j\in\mathcal{C}_2} z_j = \mathbf{w}^T(\mathbf{m}_1 - \mathbf{m}_2). \tag{22.3}$$

Các phương sai nội lớp được định nghĩa bởi

$$s_k^2 = \sum_{n\in\mathcal{C}_k}(z_n - m_k)^2, \quad k = 1, 2 \tag{22.4}$$

Chú ý: Các phương sai nội lớp không được lấy trung bình như phương sai thông thường. Điều này có thể lý giải bởi tầm quan trọng của mỗi phương sai nội lớp cần tỉ lệ thuận với số lượng điểm dữ liệu trong lớp đó, tức phương sai nội lớp bằng phương sai nhân với số điểm trong lớp đó.

LDA là thuật toán đi tìm giá trị lớn nhất của hàm mục tiêu:

$$J(\mathbf{w}) = \frac{(m_1 - m_2)^2}{s_1^2 + s_2^2} \tag{22.5}$$

Thông qua việc tối đa hàm mục tiêu này, ta sẽ thu được phương sai liên lớp $(m_1 - m_2)^2$ lớn và phương sai nội lớp $s_1^2 + s_2^2$ nhỏ.

Tiếp theo, chúng ta sẽ tìm biểu thức phụ thuộc giữa tử số và mẫu số trong vế phải của (22.5) vào $\mathbf{w}$. Tử số được viết lại thành:

$$(m_1 - m_2)^2 = (\mathbf{w}^T(\mathbf{m}_1 - \mathbf{m}_2))^2 = \mathbf{w}^T \underbrace{(\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T}_{\mathbf{S}_B} \mathbf{w} = \mathbf{w}^T \mathbf{S}_B \mathbf{w}$$

$\mathbf{S}_B$ còn được gọi là *ma trận phương sai liên lớp*. Có thể thấy đây là một ma trận đối xứng nửa xác định dương.

Mẫu số được viết lại thành:

$$\begin{aligned} s_1^2 + s_2^2 &= \sum_{k=1}^{2} \sum_{n \in \mathcal{C}_k} \left(\mathbf{w}^T(\mathbf{x}_n - \mathbf{m}_k)\right)^2 \\ &= \mathbf{w}^T \underbrace{\sum_{k=1}^{2} \sum_{n \in \mathcal{C}_k} (\mathbf{x}_n - \mathbf{m}_k)(\mathbf{x}_n - \mathbf{m}_k)^T}_{\mathbf{S}_W} \mathbf{w} = \mathbf{w}^T \mathbf{S}_W \mathbf{w} \end{aligned} \tag{22.6}$$

$\mathbf{S}_W$ còn được gọi là *ma trận phương sai nội lớp*. Đây là một ma trận đối xứng nửa xác định dương vì nó là tổng của hai ma trận đối xứng nửa xác định dương[57].

Như vậy, bài toán tối ưu cho LDA trở thành

$$\mathbf{w} = \arg\max_{\mathbf{w}} \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}} \tag{22.7}$$

22.2.3. Nghiệm của bài toán tối ưu

Nghiệm $\mathbf{w}$ của (22.7) là nghiệm của phương trình đạo hàm hàm mục tiêu bằng không. Sử dụng quy tắc chuỗi cho đạo hàm nhiều biến và công thức $\nabla_{\mathbf{w}} \mathbf{w}\mathbf{A}\mathbf{w} = 2\mathbf{A}\mathbf{w}$ với $\mathbf{A}$ là một ma trận đối xứng, ta thu được:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{(\mathbf{w}^T \mathbf{S}_W \mathbf{w})^2} \left(2\mathbf{S}_B \mathbf{w}(\mathbf{w}^T \mathbf{S}_W \mathbf{w}) - 2\mathbf{w}^T \mathbf{S}_B \mathbf{w}^T \mathbf{S}_W \mathbf{w}\right) = \mathbf{0} \tag{22.8}$$

$$\Leftrightarrow \mathbf{S}_B \mathbf{w} = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}} \mathbf{S}_W \mathbf{w} = J(\mathbf{w}) \mathbf{S}_W \mathbf{w} \tag{22.9}$$

$$\Rightarrow \mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = J(\mathbf{w}) \mathbf{w} \tag{22.10}$$

[57] $(\mathbf{a}^T\mathbf{b})^2 = (\mathbf{a}^T\mathbf{b})(\mathbf{a}^T\mathbf{b}) = \mathbf{a}^T\mathbf{b}\mathbf{b}^T\mathbf{a}$ với $\mathbf{a}, \mathbf{b}$ là hai vector cùng chiều bất kỳ.

Lưu ý: Trong (22.10), ta đã giả sử rằng ma trận $\mathbf{S}_W$ khả nghịch. Điều này không luôn đúng, nhưng có thể được khắc phục bằng một kỹ thuật nhỏ là xấp xỉ $\mathbf{S}_W$ bởi $\bar{\mathbf{S}}_W = \mathbf{S}_W + \lambda\mathbf{I}$ với λ là một số thực dương nhỏ. Ma trận mới này khả nghịch vì trị riêng nhỏ nhất của nó dương, bằng với trị riêng nhỏ nhất của $\mathbf{S}_W$ cộng với λ. Điều này suy ra từ việc $\mathbf{S}_W$ là một ma trận nửa xác định dương. Từ đó, $\bar{\mathbf{S}}_W$ là một ma trận xác định dương vì mọi trị riêng của nó không nhỏ hơn λ, và vì vậy khả nghịch. Khi tính toán, ta có thể sử dụng nghịch đảo của $\bar{\mathbf{S}}_W$. Kỹ thuật này được sử dụng rất nhiều khi cần sử dụng nghịch đảo của một ma trận nửa xác định dương và chưa biết nó có thực sự xác định dương hay không.

Trở lại đẳng thức (22.10), vì $J(\mathbf{w})$ là một số vô hướng, ta suy ra $\mathbf{w}$ phải là một vector riêng của $\mathbf{S}_W^{-1}\mathbf{S}_B$ ứng với $J(\mathbf{w})$. Vậy, để hàm mục tiêu là lớn nhất thì $J(\mathbf{w})$ phải là trị riêng lớn nhất của $\mathbf{S}_W^{-1}\mathbf{S}_B$. Dấu bằng xảy ra khi $\mathbf{w}$ là vector riêng ứng với trị riêng lớn nhất đó. Từ đó, nếu $\mathbf{w}$ là nghiệm của (22.7) thì $k\mathbf{w}$ cũng là nghiệm với k là số thực khác không bất kỳ. Vậy ta có thể chọn $\mathbf{w}$ sao cho $(\mathbf{m}_1 - \mathbf{m}_2)^T\mathbf{w} = L$ với L là trị riêng lớn nhất của $\mathbf{S}_W^{-1}\mathbf{S}_B$ và cũng là giá trị tối ưu của $J(\mathbf{w})$. Khi đó, thay định nghĩa của $\mathbf{S}_B$ ở (22.6) vào (22.10) ta có:

$$L\mathbf{w} = \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)\underbrace{(\mathbf{m}_1 - \mathbf{m}_2)^T\mathbf{w}}_{L} = L\mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2) \tag{22.11}$$

Điều này nghĩa là ta có thể chọn

$$\mathbf{w} = \alpha\mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2) \tag{22.12}$$

với $\alpha \neq 0$ bất kỳ. Biểu thức (22.12) còn được biết với tên gọi *biệt thức tuyến tính Fisher* (Fisher's linear discriminant), được đặt theo tên nhà khoa học Ronald Fisher (https://goo.gl/eUk1KS).

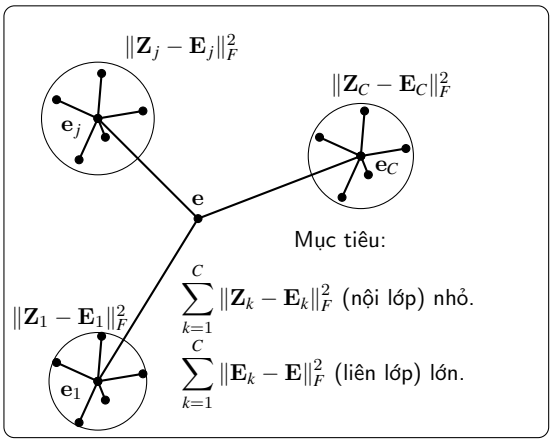
22.3. Bài toán phân loại đa lớp

22.3.1. Xây dựng hàm mục tiêu

Trong mục này, chúng ta sẽ xem xét trường hợp tổng quát của LDA khi có nhiều hơn hai lớp dữ liệu, $C > 2$. Giả sử rằng chiều của dữ liệu D lớn hơn C. Đặt số chiều dữ liệu mới là $D' < D$ và dữ liệu mới ứng với mỗi điểm dữ liệu $\mathbf{x}$ là:

$$\mathbf{z} = \mathbf{W}^T\mathbf{x} \tag{22.13}$$

với $\mathbf{W} \in \mathbb{R}^{D\times D'}$.



Hình 22.3. LDA cho bài toán phân loại đa lớp. Mục đích chung là phương sai nội lớp nhỏ và phương sai liên lớp lớn. Các vòng tròn thể hiện các lớp khác nhau.

Một vài ký hiệu:

- $\mathbf{X}_k, \mathbf{Z}_k = \mathbf{W}^T\mathbf{X}_k$ lần lượt là ma trận dữ liệu của lớp thứ k trong không gian ban đầu và không gian mới với số chiều nhỏ hơn. Mỗi cột tương ứng với một điểm dữ liệu.
- $\mathbf{m}_k = \dfrac{1}{N_k}\sum_{n\in\mathcal{C}_k} \mathbf{x}_k \in \mathbb{R}^D$ là vector trung bình của lớp k trong không gian ban đầu.
- $\mathbf{e}_k = \dfrac{1}{N_k}\sum_{n\in\mathcal{C}_k} \mathbf{z}_n = \mathbf{W}^T\mathbf{m}_k \in \mathbb{R}^{D'}$ là vector trung bình của lớp k trong không gian mới.
- $\mathbf{m} \in \mathbb{R}^D$ là vector trung bình của toàn bộ dữ liệu trong không gian ban đầu và $\mathbf{e} \in R^{D'}$ là vector trung bình trong không gian mới.

Một trong những cách xây dựng hàm mục tiêu cho LDA đa lớp được minh họa trong Hình 22.3. Độ phân tán của một lớp dữ liệu được coi như tổng bình phương khoảng cách từ mỗi điểm trong lớp đó tới vector trung bình của chúng. Nếu tất cả các điểm đều gần vector trung bình này thì tập dữ liệu đó có độ phân tán nhỏ. Ngược lại, nếu tổng này lớn, tức trung bình các điểm đều xa trung tâm, tập hợp này được coi là có độ phân tán cao. Dựa vào nhận xét này, ta có thể xây dựng các đại lượng phương sai nội lớp và liên lớp như sau đây.

Phương sai nội lớp của lớp thứ k được định nghĩa như sau:

$$\sigma_k^2 = \sum_{n\in\mathcal{C}_k} \|\mathbf{z}_n - \mathbf{e}_k\|_F^2 = \|\mathbf{Z}_k - \mathbf{E}_k\|_2^2 = \|\mathbf{W}^T(\mathbf{X}_k - \mathbf{M}_k)\|_F^2 \tag{22.14}$$

$$= \text{trace}\left(\mathbf{W}^T(\mathbf{X}_k - \mathbf{M}_k)(\mathbf{X}_k - \mathbf{M}_k)^T\mathbf{W}\right) \tag{22.15}$$

Với $\mathbf{E}_k$ là một ma trận có các cột giống hệt nhau và bằng với vector trung bình $\mathbf{e}_k$. Có thể nhận thấy $\mathbf{E}_k = \mathbf{W}^T\mathbf{M}_k$ với $\mathbf{M}_k$ là ma trận có các cột giống hệt nhau và bằng với vector trung bình $\mathbf{m}_k$ trong không gian ban đầu. Vậy đại lượng đo phương sai nội lớp trong LDA đa lớp có thể được đo bằng:

$$s_W = \sum_{k=1}^{C} \sigma_k^2 = \sum_{k=1}^{C} \text{trace}\left(\mathbf{W}^T(\mathbf{X}_k - \mathbf{M}_k)(\mathbf{X}_k - \mathbf{M}_k)^T\mathbf{W}\right) = \text{trace}\left(\mathbf{W}^T\mathbf{S}_W\mathbf{W}\right)$$

với

$$\mathbf{S}_W = \sum_{k=1}^{C} \|\mathbf{X}_k - \mathbf{M}_k\|_F^2 = \sum_{k=1}^{C} \sum_{n\in\mathcal{C}_k} (\mathbf{x}_n - \mathbf{m}_k)(\mathbf{x}_n - \mathbf{m}_k)^T \tag{22.16}$$

Ma trận $\mathbf{S}_W$ này là một ma trận nửa xác định dương.

Phương sai liên lớp lớn có thể đạt được nếu tất cả các điểm trong không gian mới đều xa vector trung bình chung $\mathbf{e}$. Điều này cũng có thể đạt được nếu các vector trung bình của mỗi lớp xa các vector trung bình chung (trong không gian mới). Vậy ta có thể định nghĩa đại lượng phương sai liên lớp như sau:

$$s_B = \sum_{k=1}^{C} N_k\|\mathbf{e}_k - \mathbf{e}\|_F^2 = \sum_{k=1}^{C} \|\mathbf{E}_k - \mathbf{E}\|_F^2 \tag{22.17}$$

Ta lấy N_k làm trọng số vì có thể có những lớp có nhiều phần tử so với các lớp còn lại. Chú ý rằng ma trận $\mathbf{E}$ có thể có số cột linh động, phụ thuộc vào số cột của ma trận $\mathbf{E}_k$ mà nó đi cùng (và bằng N_k).

Lập luận tương tự như (22.16), bạn đọc có thể chứng minh được:

$$s_B = \text{trace}\left(\mathbf{W}^T\mathbf{S}_B\mathbf{W}\right) \tag{22.18}$$

với

$$\mathbf{S}_B = \sum_{k=1}^{C} (\mathbf{M}_k - \mathbf{M})(\mathbf{M}_k - \mathbf{M})^T = \sum_{k=1}^{C} N_k(\mathbf{m}_k - \mathbf{m})(\mathbf{m}_k - \mathbf{m})^T \tag{22.19}$$

và số cột của ma trận $\mathbf{M}$ cũng linh động theo số cột của $\mathbf{M}_k$. Ma trận này một ma trận nửa đối xứng xác định dương vì nó là tổng của các ma trận đối xứng nửa xác định dương.

22.3.2. Hàm mục tiêu của LDA đa lớp

Với cách định nghĩa và ý tưởng về phương sai nội lớp nhỏ và phương sai đa lớp lớn như trên, ta có thể xây dựng bài toán tối ưu:

$$\mathbf{W} = \arg\max_{\mathbf{W}} J(\mathbf{W}) = \arg\max_{\mathbf{W}} \frac{\text{trace}(\mathbf{W}^T\mathbf{S}_B\mathbf{W})}{\text{trace}(\mathbf{W}^T\mathbf{S}_W\mathbf{W})} \tag{22.20}$$

Nghiệm của bài toán tối ưu này được tìm bằng cách giải phương trình đạo hàm hàm mục tiêu bằng không. Nhắc lại về đạo hàm của hàm trace theo ma trận:

$$\nabla_{\mathbf{W}}\text{trace}(\mathbf{W}^T\mathbf{A}\mathbf{W}) = 2\mathbf{A}\mathbf{W} \tag{22.21}$$

với $\mathbf{A} \in \mathbb{R}^{D\times D}$ là một ma trận đối xứng.

Với cách tính tương tự như (22.8) - (22.10), ta có:

$$\begin{aligned}
\nabla_{\mathbf{W}} J(\mathbf{W}) &= \frac{2\left(\mathbf{S}_B\mathbf{W}\text{trace}(\mathbf{W}^T\mathbf{S}_W\mathbf{W}) - \text{trace}(\mathbf{W}^T\mathbf{S}_B\mathbf{W})\mathbf{S}_W\mathbf{W}\right)}{\left(\text{trace}(\mathbf{W}^T\mathbf{S}_W\mathbf{W})\right)^2} = \mathbf{0} \\
\Leftrightarrow \mathbf{S}_W^{-1}\mathbf{S}_B\mathbf{W} &= \frac{\text{trace}(\mathbf{W}^T\mathbf{S}_B\mathbf{W})}{\text{trace}(\mathbf{W}^T\mathbf{S}_W\mathbf{W})}\mathbf{W} = J(\mathbf{W})\mathbf{W}
\end{aligned}$$

Từ đó suy ra mỗi cột của $\mathbf{W}$ là một vector riêng của $\mathbf{S}_W^{-1}\mathbf{S}_B$ ứng với trị riêng lớn nhất của ma trận này. Nhận thấy rằng các cột của $\mathbf{W}$ phải độc lập tuyến tính. Vì nếu không, dữ liệu trong không gian mới $\mathbf{z} = \mathbf{W}^T\mathbf{x}$ sẽ phụ thuộc tuyến tính và có thể tiếp tục được giảm số chiều. Vậy các cột của $\mathbf{W}$ là các vector độc lập tuyến tính ứng với trị riêng cao nhất của $\mathbf{S}_W^{-1}\mathbf{S}_B$. Câu hỏi đặt ra là có nhiều nhất bao nhiêu vector riêng độc lập tuyến tính ứng với trị riêng lớn nhất của $\mathbf{S}_W^{-1}\mathbf{S}_B$? Số lượng này chính là số chiều D' của dữ liệu mới.

Số lượng lớn nhất các vector riêng độc lập tuyến tính ứng với một trị riêng của một ma trận không thể lớn hơn hạng của ma trận đó. Dưới đây là một bổ đề quan trọng.

Bổ đề:

$$\text{rank}(\mathbf{S}_B) \leq C - 1 \tag{22.22}$$

Chứng minh[58]:

Viết lại 22.19 dưới dạng

$$\mathbf{S}_B = \mathbf{P}\mathbf{P}^T \tag{22.23}$$

với $\mathbf{P} \in R^{D\times C}$ mà cột thứ k cuả nó là $\mathbf{p}_k = \sqrt{N_k}(\mathbf{m}_k - \mathbf{m})$.

Cột cuối cùng là một tổ hợp tuyến tính của các cột còn lại vì

$$\mathbf{m}_C - \mathbf{m} = \mathbf{m}_C - \frac{\sum_{k=1}^{C} N_k\mathbf{m}_k}{N} = \sum_{k=1}^{C-1}\frac{N_k}{N}(\mathbf{m}_k - \mathbf{m}) \tag{22.24}$$

Như vậy ma trận $\mathbf{P}$ có nhiều nhất $C-1$ cột độc lập tuyến tính, vì vậy hạng[59] của nó không vượt quá $C-1$. Cuối cùng, $\mathbf{S}_B$ là tích của hai ma trận với hạng không quá $C-1$, nên hạng của nó không vượt quá $C-1$.

[58] Việc chứng minh này không thực sự quan trọng, chỉ phù hợp với những bạn muốn hiểu sâu.
[59] Các tính chất của hạng có thể được tìm thấy trong Mục 1.8.

Từ đó ra có rank $\left(\mathbf{S}_W^{-1}\mathbf{S}_B\right) \leq \text{rank}\mathbf{S}_B \leq C-1$. Vậy số chiều của không gian mới là một số không lớn hơn $C-1$.

Tóm lại, nghiệm của bài toán multi-class LDA là các vector riêng độc lập tuyến tính ứng với trị riêng cao nhất của $\mathbf{S}_W^{-1}\mathbf{S}_B$.

Lưu ý: Có nhiều cách khác nhau để xây dựng hàm mục tiêu cho LDA đa lớp dựa trên việc định nghĩa phương sai nội lớp nhỏ và phương sai liên lớp lớn. Chúng ta đang sử dụng hàm trace để đong đếm hai đại lượng này. Một ví dụ khác về hàm tối ưu là $J(\mathbf{W}) = \text{trace}(s_W^{-1}s_B) = \text{trace}\{(\mathbf{W}\mathbf{S}_W\mathbf{W}^T)^{-1}(\mathbf{W}\mathbf{S}_B\mathbf{W}^T)\}$ [Fuk13]. Hàm số này cũng đạt giá trị lớn nhất khi $\mathbf{W}$ là tập hợp của D' vector riêng ứng với các trị riêng lớn nhất của $\mathbf{S}_W^{-1}\mathbf{S}_B$. Có một điểm chung giữa các cách tiếp cận này là chiều của không gian mới không vượt quá $C-1$.

22.4. Ví dụ trên Python

Trong mục này, chúng ta sẽ minh hoạ LDA cho bài toán phân loại nhị phân qua một ví dụ đơn giản với dữ liệu trong không gian hai chiều.

Dữ liệu của hai lớp được tạo như sau:

```
from __future__ import division, print_function, unicode_literals
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages
np.random.seed(22)

means = [[0, 5], [5, 0]]
cov0 = [[4, 3], [3, 4]]
cov1 = [[3, 1], [1, 1]]
N0, N1 = 50, 40
N = N0 + N1
X0 = np.random.multivariate_normal(means[0], cov0, N0) # each row is a point
X1 = np.random.multivariate_normal(means[1], cov1, N1)
```

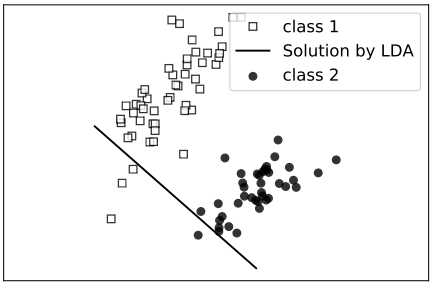
Hai lớp dữ liệu được minh hoạ bởi các điểm hình vuông và tròn trong Hình 22.4. Tiếp theo, chúng ta tính các ma trận phương sai nội lớp và đa lớp:

```
# Build S_B
m0 = np.mean(X0.T, axis = 1, keepdims = True)
m1 = np.mean(X1.T, axis = 1, keepdims = True)

a = (m0 - m1)
S_B = a.dot(a.T)

# Build S_W
A = X0.T - np.tile(m0, (1, N0))
B = X1.T - np.tile(m1, (1, N1))

S_W = A.dot(A.T) + B.dot(B.T)
```



Hình 22.4. Ví dụ minh hoạ về LDA trong không gian hai chiều. Dữ liệu sẽ được chiếu lên đường thẳng. Nếu chiếu lên đường thẳng này, dữ liệu của hai lớp sẽ nằm về hai phía của một điểm trên đường thẳng đó.

Nghiệm của bài toán là vector riêng ứng với trị riêng lớn nhất của $\mathbf{S}_W^{-1}\mathbf{W}_B$:

```
_, W = np.linalg.eig(np.linalg.inv(S_W).dot(S_B))
w = W[:,0]
print('w = ', w)
```

Kết quả:

```
w =  [ 0.75091074 -0.66040371]
```

Đường thẳng có phương `w` được minh hoạ trong Hình 22.4. Ta thấy rằng nghiệm này hợp lý với dữ liệu của bài toán.

Để kiểm chứng độ chính xác của nghiệm tìm được, ta cùng so sánh nó với nghiệm tìm được bởi thư viện `sklearn`:

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X = np.concatenate((X0, X1))
y = np.array([0]*N0 + [1]*N1)
clf = LinearDiscriminantAnalysis()
clf.fit(X, y)

print('w_sklearn = ', clf.coef_[0]/np.linalg.norm(clf.coef_)) # normalize
```

```
w_sklearn =  [ 0.75091074 -0.66040371]
```

Nghiệm tìm theo công thức và nghiệm tìm theo thư viện là như nhau.

Một ví dụ khác so sánh PCA và LDA có thể được tìm thấy tại *Comparison of LDA and PCA 2D projection of Iris dataset* (https://goo.gl/tWjAEs).

22.5. Thảo luận

- LDA là một phương pháp giảm chiều dữ liệu có sử dụng thông tin về label của dữ liệu. Vì vậy, LDA là một thuật toán học có giám sát.

- Ý tưởng cơ bản của LDA là tìm một không gian mới với số chiều nhỏ hơn không gian ban đầu sao cho hình chiếu của các điểm trong cùng lớp trong không gian mới gần nhau trong khi hình chiếu của các điểm thuộc các lớp khác nhau xa nhau.

- Trong PCA, số chiều của không gian mới có thể là bất kỳ số nào không lớn hơn số chiều và số điểm dữ liệu. Trong LDA, với bài toán có C lớp, số chiều của không gian mới không được vượt quá $C - 1$.

- Với bài toán có hai lớp, từ Hình 22.1 có thể thấy rằng hai lớp là tách biệt tuyến tính khi và chỉ khi tồn tại một đường thẳng và một điểm trên đường thẳng đó sao cho dữ liệu hình chiếu của hai lớp nằm về hai phía khác nhau của điểm đó.

- LDA hoạt động rất tốt nếu các lớp là tách biệt tuyến tính. Chất lượng mô hình giảm đi rõ rệt nếu các lớp không tách biệt tuyến tính. Điều này dễ hiểu vì dữ liệu chiếu lên mọi phương vẫn bị chồng lần, và việc tách biệt không thể thực hiện được như ở không gian ban đầu.

- Mặc dù LDA có nhiều hạn chế, ý tưởng về phương sai nội lớp nhỏ và phương sai liên lớp lớn được sử dụng rất nhiều trong các thuật toán phân loại [VM17, VM16, YZFZ11].